Neural Networks

Cross Entropy Loss Note

AARHUS
UNIVERSITY

Multiclass Clasification

- Multinomial Regression
  - Linear model with softmax for K classes.







  - Red, white or rose?

One-in-K Encoding

- Representing Labels
  - First question is how do we represent labels from a set of K classes?
  - Represent labels as vectors:
    - K-dimensional vector.
    - One coordinate per class.
    - If label i, put a 1 in i'th coordinate and 0 in others.
  - Red: (1, 0, 0)
  - White: (0, 1, 0)
  - Rose: (0, 0, 1)



One-in-K Encoding

- Two classes:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, y = \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix}$
- Each y_i in {-1,1}

- K classes:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}$
- Each row of Y has a single ”1” and K-1 ”0”s.
- Want to learn Pr[y=i|x] for i=1,...,K.



Multinomial Regression

- K classes, single data point x:
  - $x = \begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Base estimates on $x^T W$:
  - $x^T W = \begin{pmatrix} x^T w_{:,1} & \cdots & x^T w_{:,K} \end{pmatrix}$
- Want to map $x^T W$ to a probability distribution.
- Inspired by sigmoid:
  - For vector $z = (z_1, \ldots, z_K)$
    - softmax(z) is a vector of probabilities.
    - softmax(z)$_j$= $\frac{e^{z_j}}{\sum_{i=1}^{K} e^{z_i}}$

Normalization so sum to 1.



Multinomial Regression

Recall one 1 pr. row, rest 0.

- K classes, n data points $x_1,\ldots,x_n$:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Assume Predict(X)=softmax(XW) gives true probabilities.
  - Likelihood of data (labels)?
    - $S=(x_1,y_1),\ldots,(x_n,y_n)$

- $\Pr[S|X,W]=\prod_{i=1}^{n}\Pr[y_i|x_i,W]=$
- $\prod_{i=1}^{n} y_i^T \mathrm{softmax}(x_i^T W)$

- Maximize over W!

Inner product with y_i extracts just entry j with $y_{i,j}=1$.



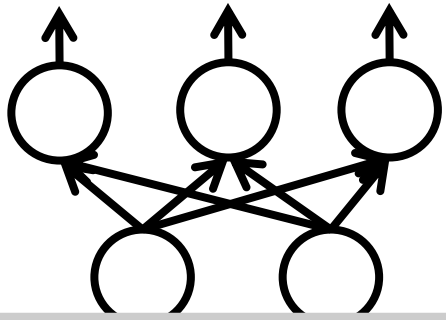
Multinomial Regression

- K classes, n data points $x_1,\ldots,x_n$:
  - $X = \begin{pmatrix} x_{1,1} & \cdots & x_{1,d} \\ \vdots & \ddots & \vdots \\ x_{n,1} & \cdots & x_{n,d} \end{pmatrix}, Y = \begin{pmatrix} y_{1,1} & \cdots & y_{1,K} \\ \vdots & \ddots & \vdots \\ y_{n,1} & \cdots & y_{n,K} \end{pmatrix}, W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,K} \\ \vdots & \ddots & \vdots \\ w_{d,1} & \cdots & w_{d,K} \end{pmatrix}$
- Maximize over W
  - $\prod_{i=1}^{n} y_i^T \text{softmax}(x_i^T W)$
- Negative log likelihood. Scale by 1/n. Minimize:
  - $E_{in}(W) = -\frac{1}{n}\sum_{i=1}^{n} \ln(y_i^T \text{softmax}(x_i^T W))$



Cross Entropy Loss

- Loss function for multiclass classification:
  - Generalizes negative log likelihood from multinomial regression.



- Setup:
  - Any machine learning model h outputting K probabilities $p_1,\ldots,p_K$ summing to 1.
    - $p_j=h(x)_j$
  - Intention $\Pr[y=j|x]=p_j$.
- Loss:
  - $L(h(x),y) = -\sum_{j=1}^{K} y_j \ln(h(x)_j)$
  - $E_{in}(h) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{K} y_{i,j}\ln(h(x_i)_j)$

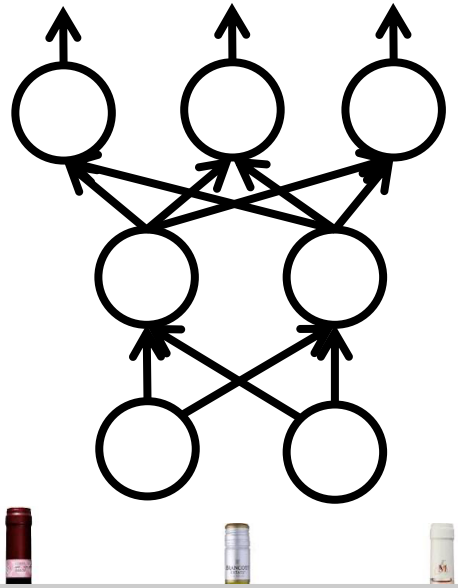
y can be any vector of probabilities summing to 1



Recall h(x) and y are length K vectors

Comparison

- Multinomial Regression:
  - $L(h(x), y) = -\ln(y^T \text{softmax}(x^T W))$
  - $E_{in}(W) = -\frac{1}{n}\sum_{i=1}^{n} \ln(y_i^T \text{softmax}(x_i^T W))$
- Cross Entropy Loss:
  - $L(h(x), y) = -\sum_{j=1}^{K} y_j \ln(h(x)_j)$
  - $E_{in}(h) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{K} y_{i,j} \ln(h(x_i)_j)$



- Recall: In multinomial regression, assumed label vector y has one 1 and rest 0. So:
  - $-\ln(y^T \text{softmax}(x^T W)) = -\sum_{j=1}^{K} y_j \ln(\text{softmax}(x^T W)_j)$
- So special case!

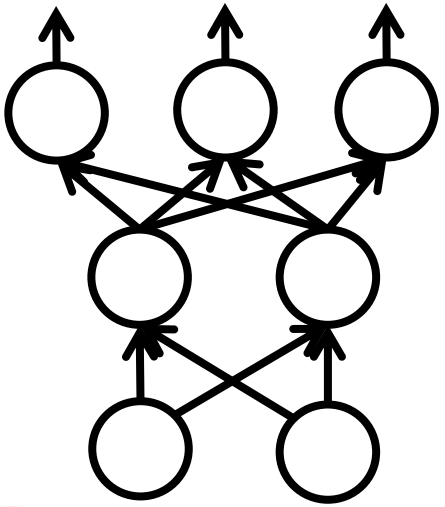
Entries of softmax $x^T W$ are probabilities summing to 1.



Cross Entropy Loss

- Loss function for multiclass classification:
  - $L(h(x), y) = -\sum_{j=1}^{K} y_j \ln(h(x)_j)$
  - $E_{in}(h) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{K} y_{i,j} \ln(h(x_i)_j)$

- Setup:
  - Any machine learning model h outputting K probabilities $p_1,\ldots,p_K$ summing to 1.
    - $p_j = h(x)_j$
  - Intention $\Pr[y=j|x] = p_j$.
- Labels y:
  - Vectors of length K. Non-negative entries summing to 1 (i.e. probability distribution).





The Name

- Recall from Decision Tree lecture

- If have n points with labels 1,…,K in a set S, then:
  - Let n_i be number of points with label i.
  - Let $p_i = n_i/n$ be fraction of points with label i.

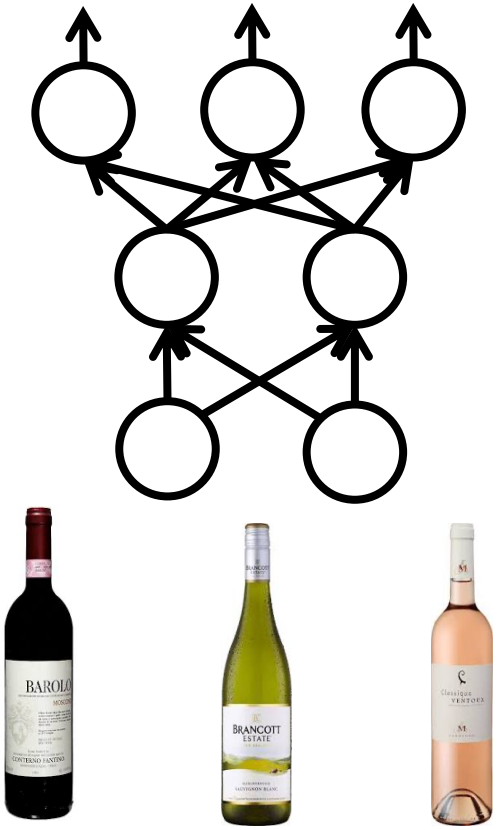
  - Entropy: $\mathrm{H(S)} = -\sum_{i=1}^{K} p_i \log_2 p_i$

- In information theory, (Shannon) entropy is a measure of uncertainty in a distribution.
- High entropy means high uncertainty.



The Name

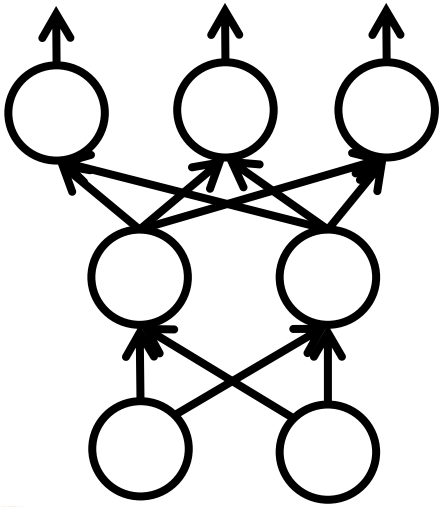
- Loss function for multiclass classification:
  - $L(h(x), y) = -\sum_{j=1}^{K} y_j \ln(h(x)_j)$
- From Decision Tree Lecture:
  - S=$y_1,\ldots,y_K$ summing to 1:
    - $H(S) = -\sum_{j=1}^{K} y_j \log_2 y_j$
- Ignoring the base of the log, cross entropy is a "crossing" of the entropy of $y_1,\ldots,y_K$ and output $h(x)_1,\ldots,h(x)_K$.



Cross Entropy Loss

- Loss function for multiclass classification:
  - $L(h(x), y) = -\sum_{j=1}^{K} y_j \ln(h(x)_j)$
  - $E_{in}(h) = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{K} y_{i,j} \ln(h(x_i)_j)$

- Setup:
  - Any machine learning model h outputting K probabilities $p_1,\ldots,p_K$ summing to 1.
    - $p_j = h(x)_j$
  - Intention $\Pr[y=j|x] = p_j$.
- Labels y:
  - Vectors of length K. Non-negative entries summing to 1 (i.e. probability distribution).

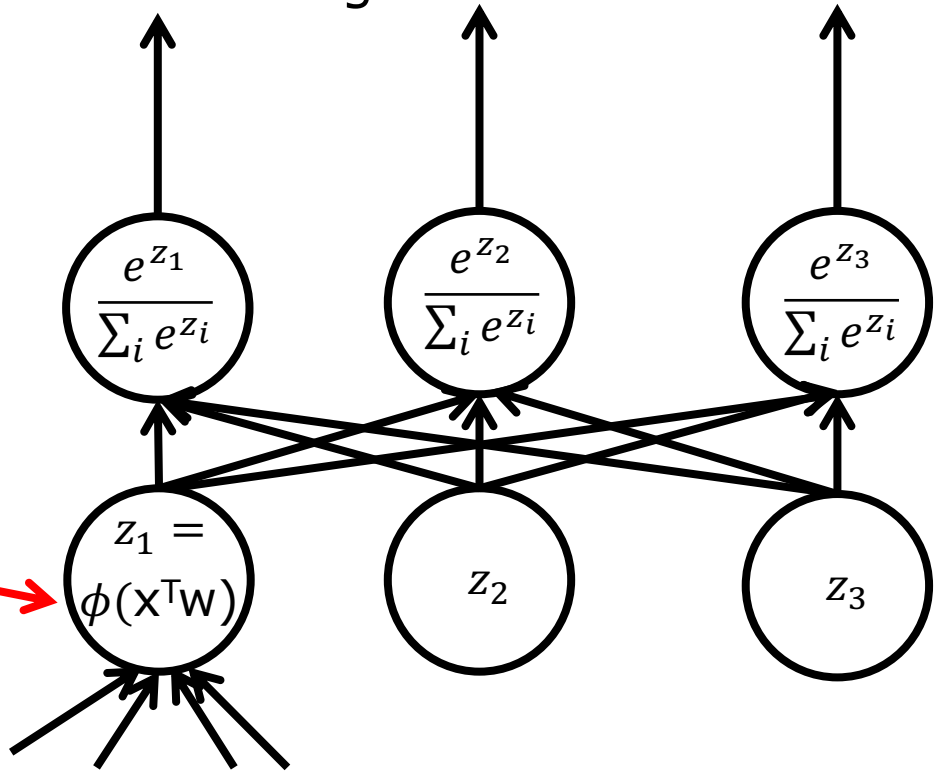




Activation Functions

- Multiclass Classication Output Neurons
  - For classification problems, often use Softmax neurons $\text{softmax}(z)_i$.
  - Output is K probabilities summing to 1.

Activation function ϕ: $R^k \rightarrow R$

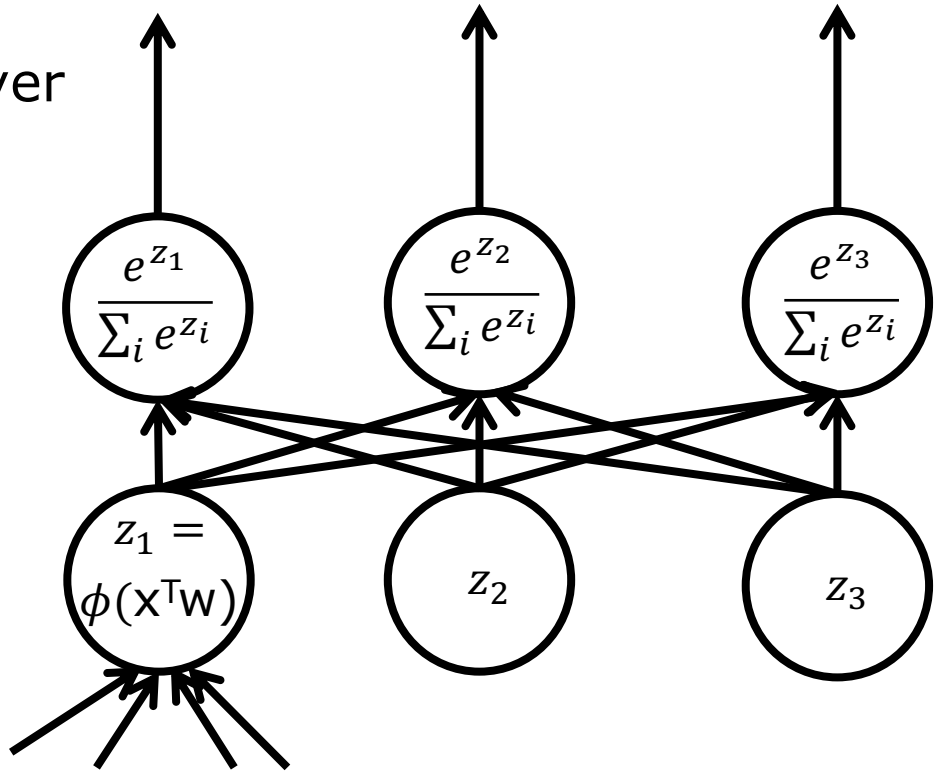


Often ϕ =Identity

Cross Entropy

- Training with Cross Entropy Loss
  - $L(h(x), y) = -\sum_{j=1}^{K} y_j \ln(h(x)_j)$

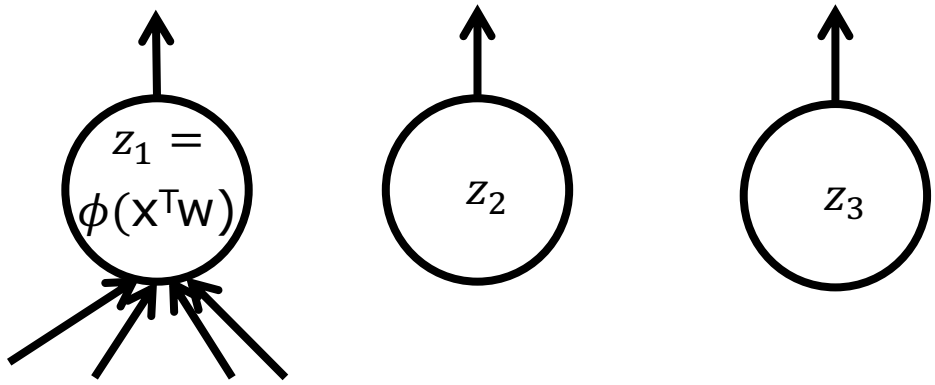
- Practical Trick:
  - Remove softmax layer and include in loss function instead.



Cross Entropy

- Training with Cross Entropy Loss
  - $L(h(x), y) = -\sum_{j=1}^{K} y_j \ln(h(x)_j)$

- Practical Trick:
  - Remove softmax layer and include in loss function instead.



Cross Entropy

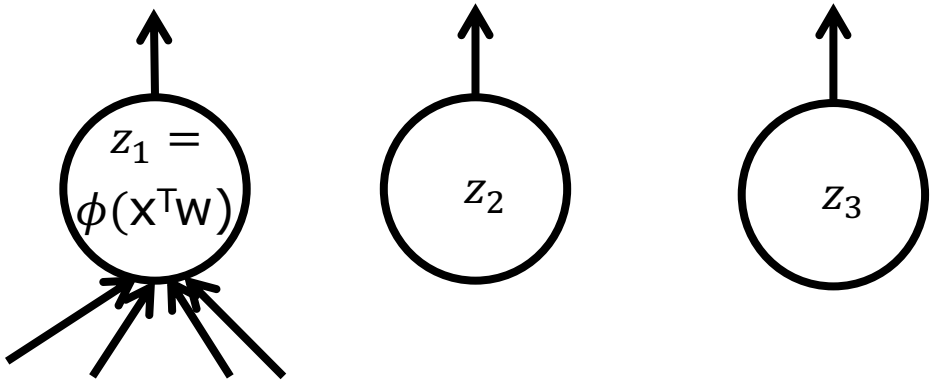
- Training with Cross Entropy Loss
  - $L(h(x), y) = -\sum_{j=1}^{K} y_j \ln(h(x)_j)$

ln(x) nicely counters the e^x in softmax.

- Practical Trick:
  - Remove softmax layer and include in loss function instead.

$$L'(h(x), y) = -\sum_{j=1}^{K} y_j \ln(softmax(z)_j)$$

Project 2: Implement neural nets with cross entropy loss for multiple classes.



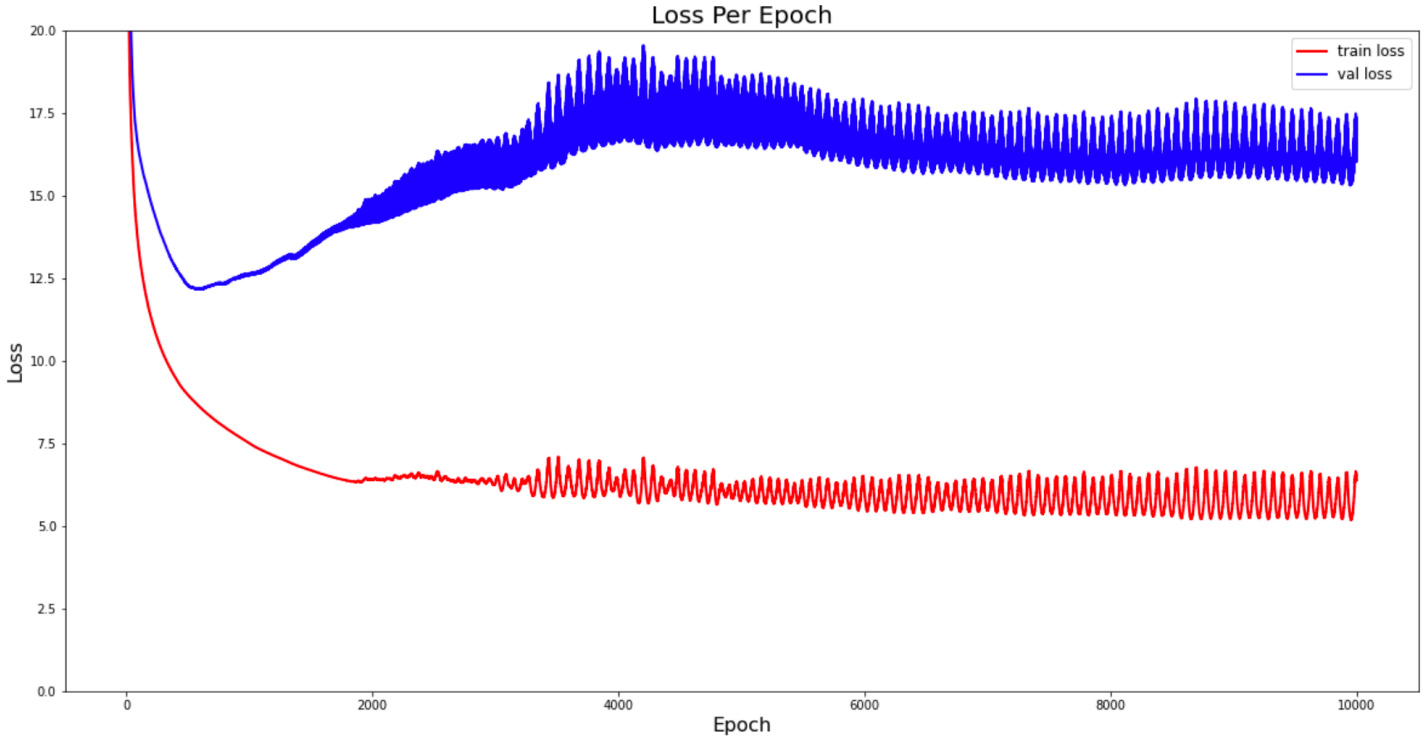
Neural Networks

[DL] 6.5 Skip 6.5.9, 6.5.10

AARHUS
UNIVERSITY

Combatting Overfitting

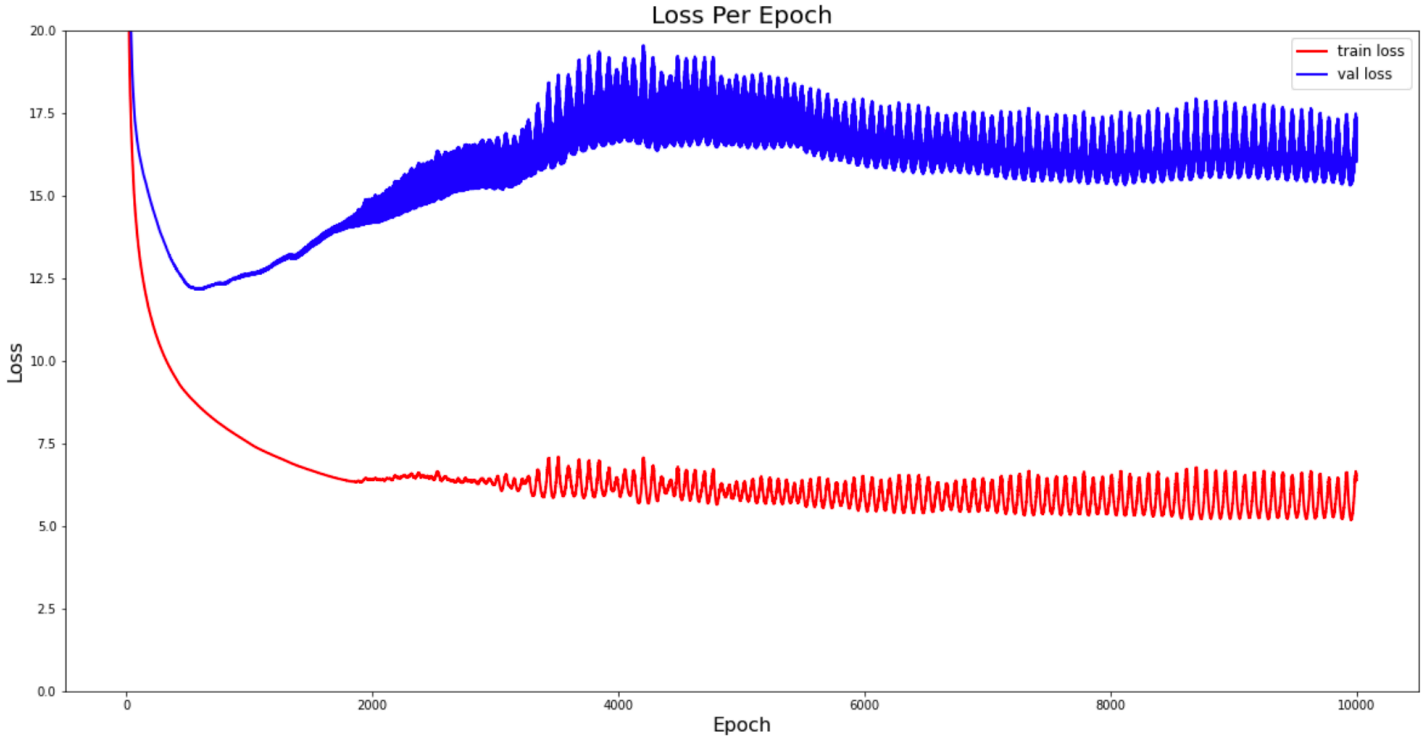
- Several Techniques:
  - Validation + Early Stopping
  - Regularization
  - Dropout (not curriculum)



Validation – Early Stopping

- Use Validation Set
- Example: Boston Housing, Regression.
  - Neural net with one ReLU hidden layer.

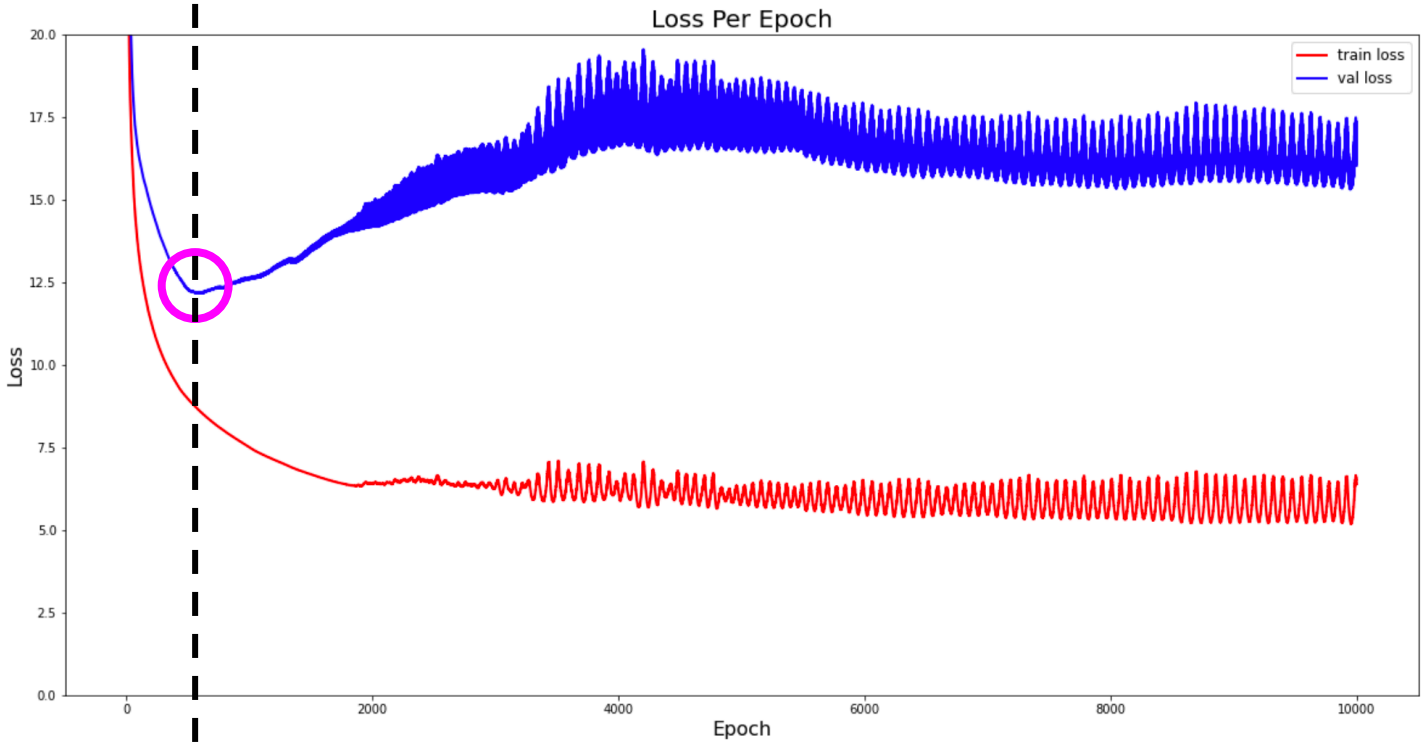
```
pytorch Neural Net Regression least squares score: 17.377429962158203
pytorch Neural Net with validation score: 17.071086883544922
```



Validation – Early Stopping

- Run 10.000 epochs of SGD:
  - E_{in} = 6.2
  - $E_{out} \approx E_{test}$ = 17.4
- Use Validation set and keep best:
  - E_{val} = 12.5, E_{in} = 8.5
  - $E_{out} \approx E_{test}$ = 17.1

```
pytorch Neural Net Regression least squares score: 17.377429962158203
pytorch Neural Net with validation score: 17.071086883544922
```



Validation – Early Stopping

- Use Validation set and keep best
  - Set aside 20% of training data as validation set D_{val}.
  - After each epoch, compute loss on D_{val}.
    - If smaller than best previous:
      - Store current model parameters.
  - After completion:
    - Use best model parameters seen.

- Early Stopping:
  - If haven't seen any improvements to best validation score for at least k epoch, stop training and use best parameters seen.



Regularization

- Weight Decay:
  - We used weight decay in linear regression (ridge regression).
  - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2 + \lambda \sum_{i=1}^{d} w_i^2$

"Normal" linear regression loss

Weight decay. Prefer small weights



Regularization

- Weight Decay:
  - We used weight decay in linear regression (ridge regression).
  - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2 + \lambda \sum_{i=1}^{d} w_i^2$
  - Also for neural nets:
  - $E_{in}(nn) = \frac{1}{n}\sum_{i=1}^{n} L(nn(x_i), y) + \lambda \sum_{w \in nn} w^2$

Remember to include in computation graph during backpropagation

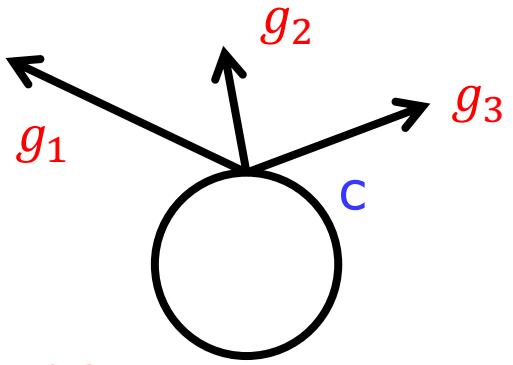
Sum over all weights in all weight matrices

Biases NOT included.



Regularization

- Weight Decay:
  - We used weight decay in linear regression (ridge regression).
  - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2 + \lambda \sum_{i=1}^{d} w_i^2$
  - Also for neural nets:
  - $E_{in}(nn) = \frac{1}{n}\sum_{i=1}^{n} L(nn(x_i), y) + \lambda \sum_{w \in nn} w^2$

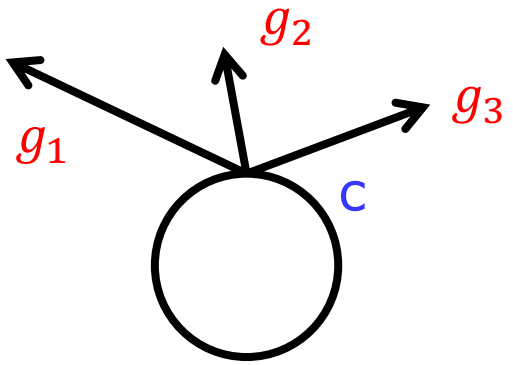


$$\frac{\partial f}{\partial c} = g_1 + g_2 + g_3$$

- Backpropagation:
  - Recall if variable branches in computation graph, must sum gradients computed along all outgoing-edges during backprop.

Regularization

- Weight Decay:
  - We used weight decay in linear regression (ridge regression).
  - $E_{in}(w) = \frac{1}{n}\sum_{i=1}^{n}\left(x_i^T w - y_i\right)^2 + \lambda \sum_{i=1}^{d} w_i^2$
  - Also for neural nets:
  - $E_{in}(nn) = \frac{1}{n}\sum_{i=1}^{n} L(nn(x_i), y) + \lambda \sum_{w \in nn} w^2$

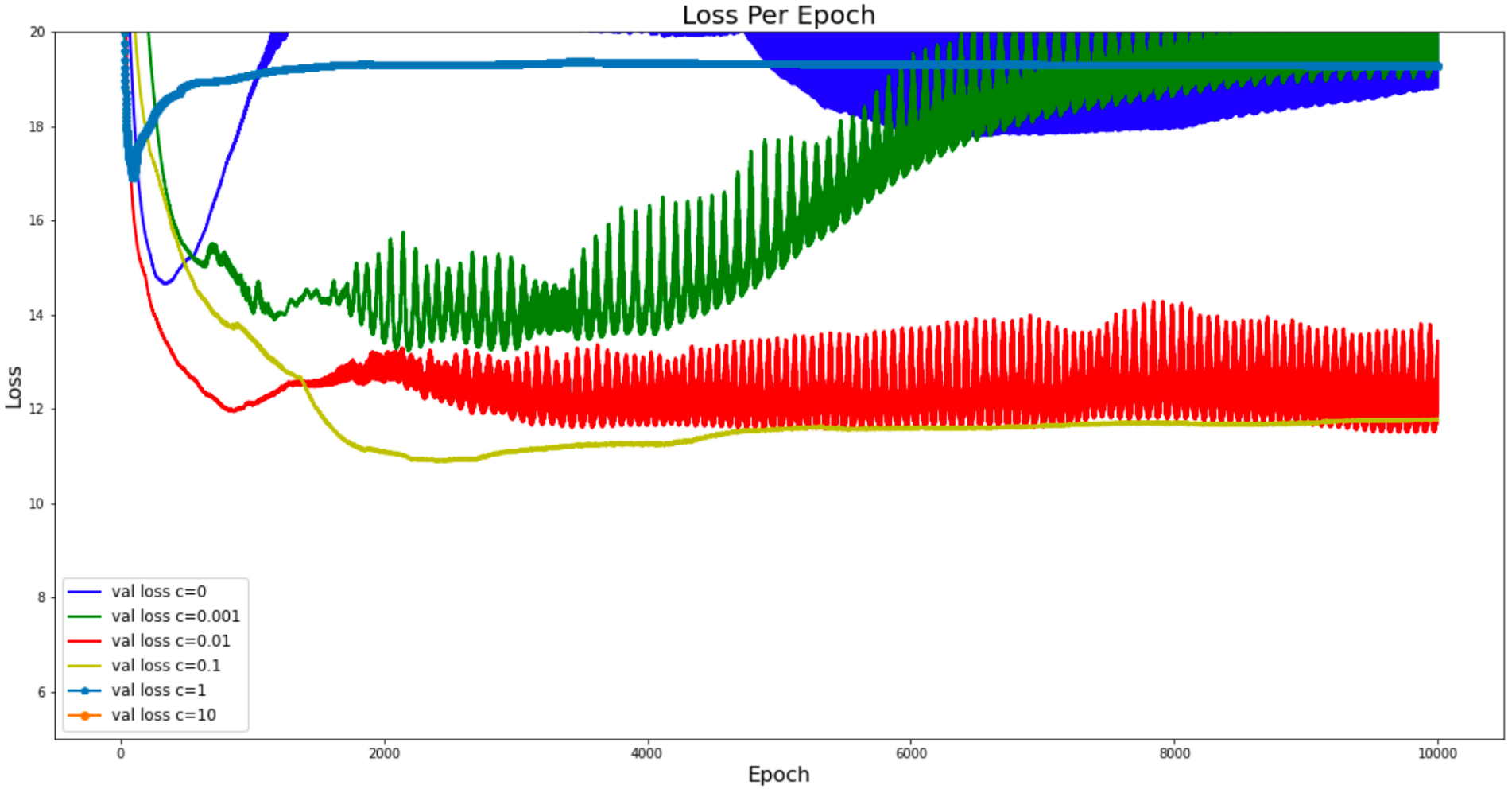


$$\frac{\partial f}{\partial c} = g_1 + g_2 + g_3$$

- Backpropagation:
  - Weight decay has simple effect:
    - Add 2λw to gradient of variable w.

Regularization

Remember: Include weight decay in loss when training, but validation must compute raw loss (no weight decay).



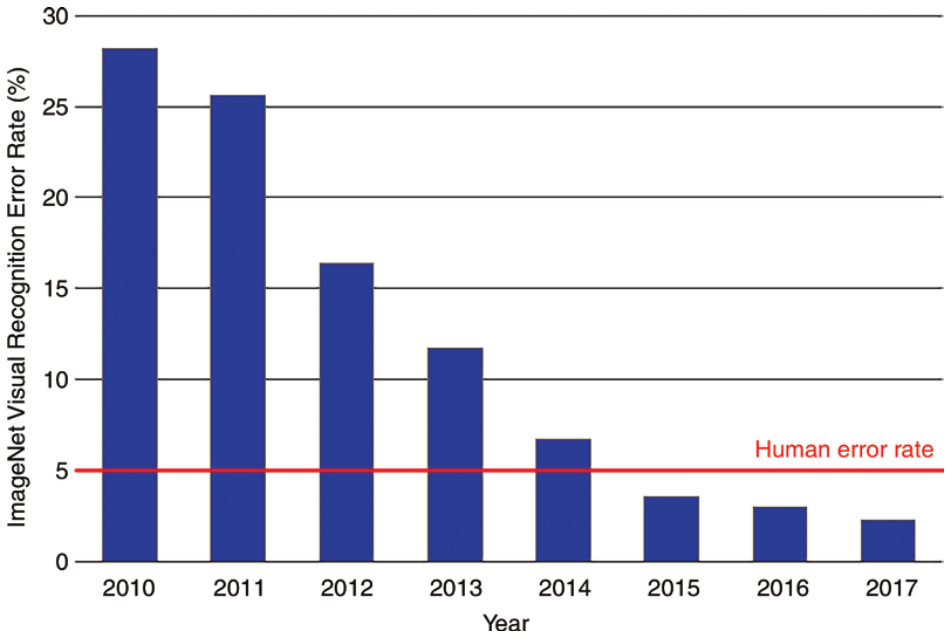
Neural Networks

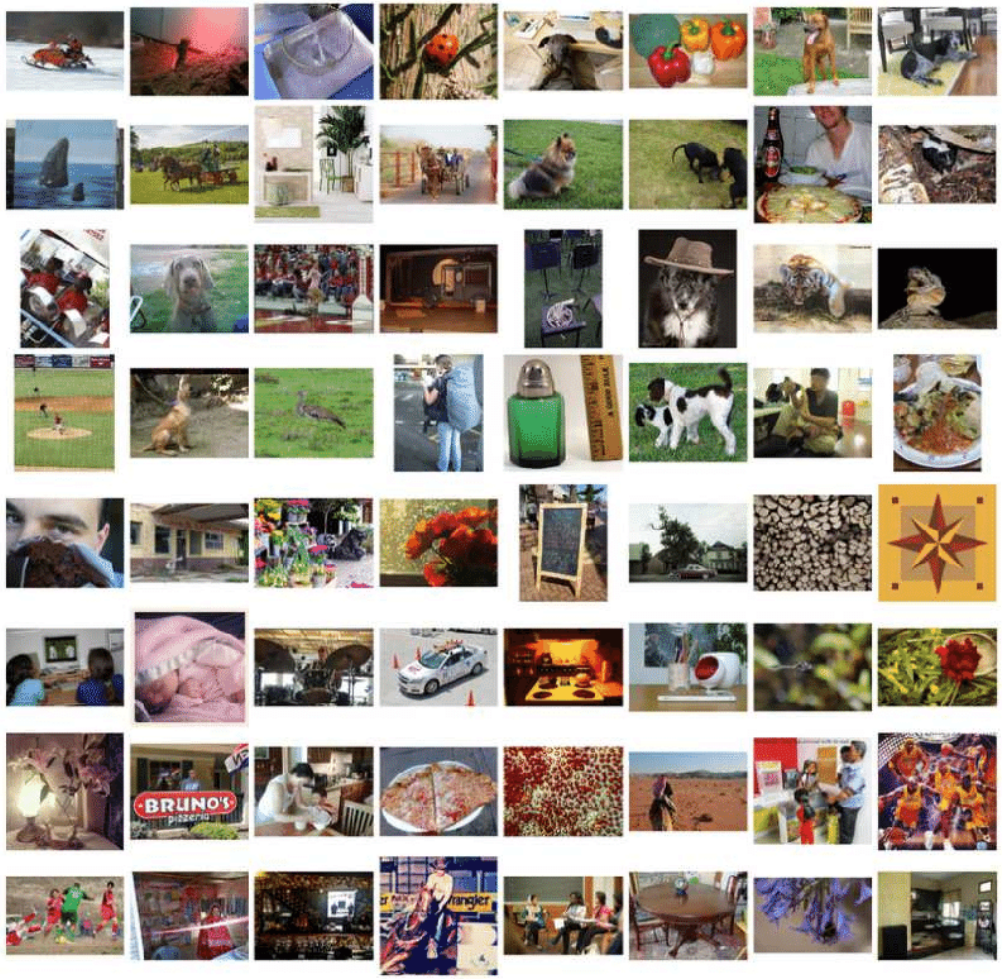
[DL] 9-9.3

AARHUS
UNIVERSITY

Deep Learning Breakthrough

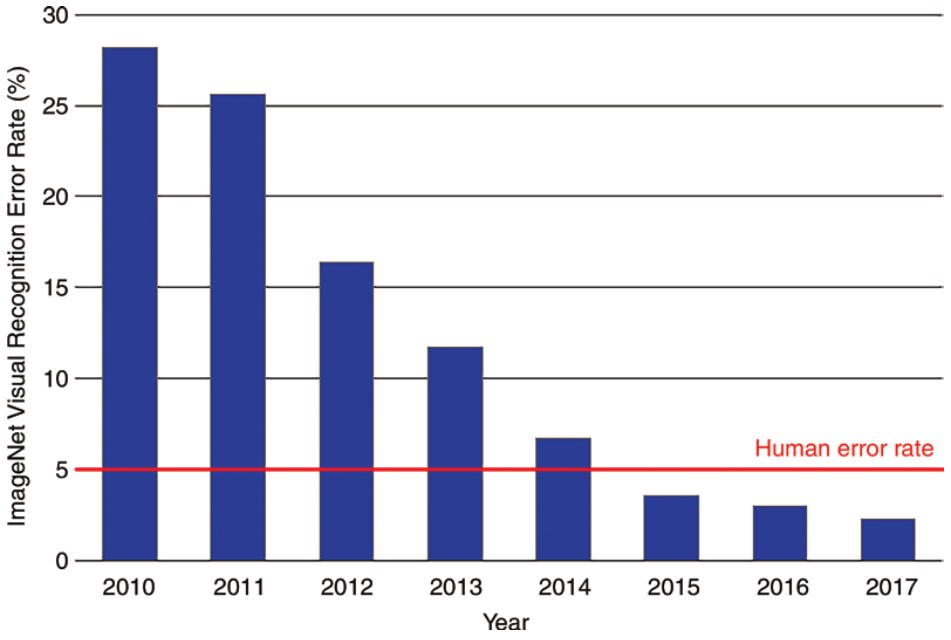
- 2012: Started using deep neural nets. AlexNet.
  - Better than humans on ImageNet data set.
  - Still not better ”in the wild”.

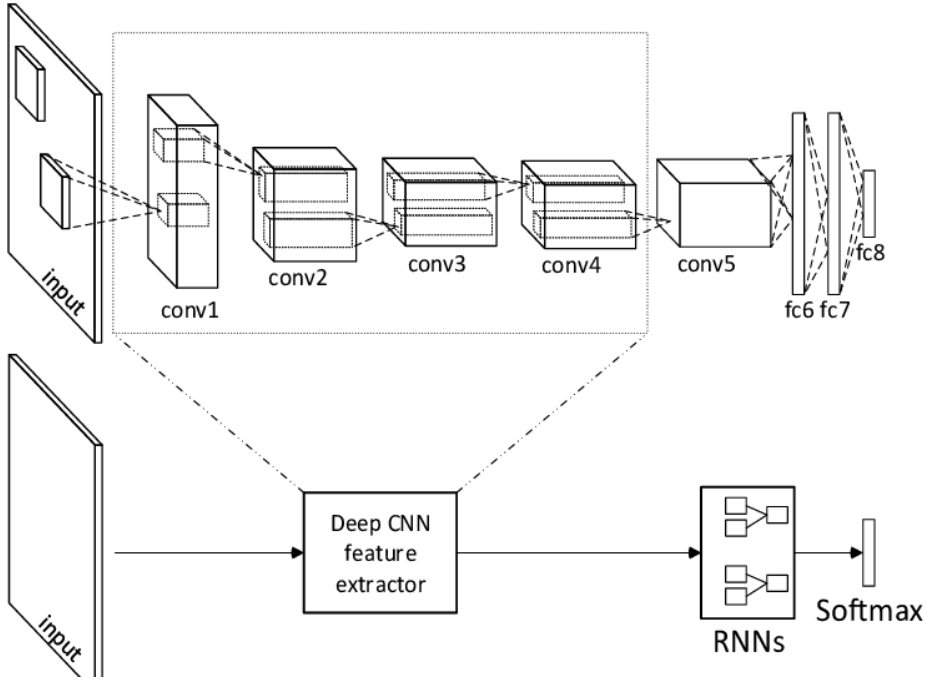




Deep Learning Breakthrough

- Image Recognition Networks:
  - Make heavy use of convolutions.

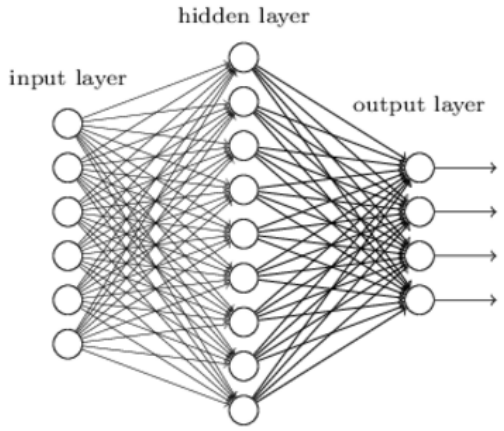




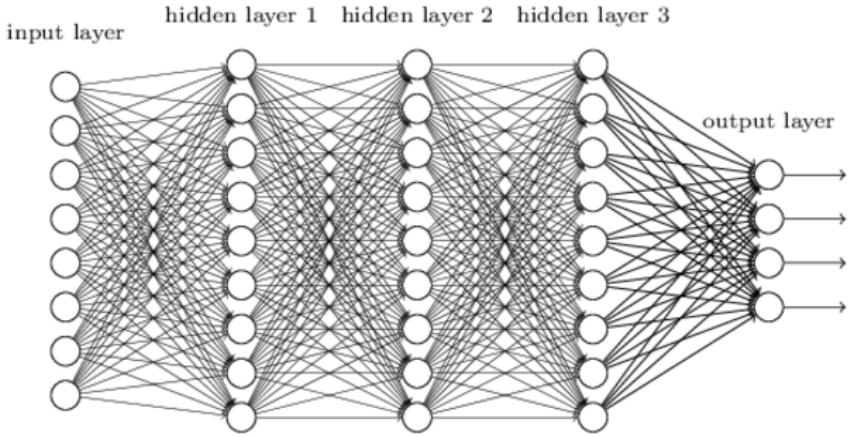
Neural Nets

- Fully Connected Feed Forward
  - Simple architecture
  - Full/dense weight matrix between each layer.
  - No sense of locality between pixels.

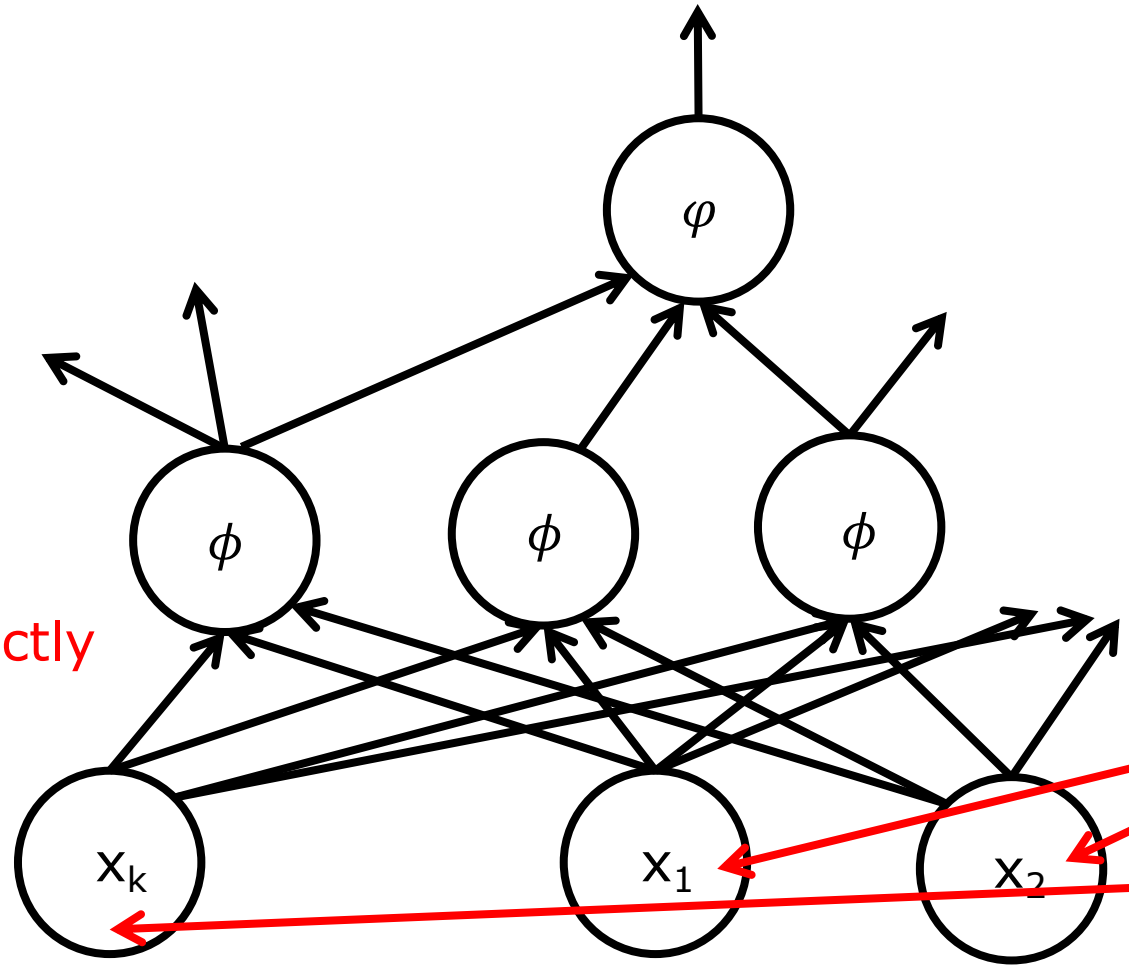
"Non-deep" feedforward neural network



Deep neural network



No Locality



Treated exactly the same

Two neighboring pixels



Convolutions

- Idea:
  - Look for small features in the image.
  - Should be translation invariant.
  - Preserve information on feature's position in image.



Cat, near right side



Cat, near left side

Convolutions

- Formally:
  - Let's look at gray-scale images. Each input neuron is a pixel with a value between 0=Black and 1=White.
  - Think of neurons as arranged in 2D grid like image.

Input neurons



0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0



Convolutions

- Formally:
  - k x k convolution specified by a k x k matrix. Often k is odd.
  - Convolution: ”Slide” convolution matrix across input.

Input

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

Convolution matrix

-1	-1	-1
-1	8	-1
-1	-1	-1



Convolutions

- Formally:
  - k x k co specified by a k x k matrix. Often k is odd.
  - Convolu e" convolution matrix across input.

-1	-1	-1
-1	8	-1
-1	-1	-1

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-0.2-0.2-0.2-0.2+2.4-0.1-0.2-0.2-0.2=0.9



Convolutions

- Formally:
  - k x k co specified by a k x k matrix. Often k is odd.
  - Convolu e" convolution matrix across input.

-1	-1	-1
-1	8	-1
-1	-1	-1

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

0.9					



Convolutions

- Formally:
  - k x k co specified by a k x k matrix. Often k is odd.
  - Convolu e" convolution matrix across input.

-1	-1	-1
-1	8	-1
-1	-1	-1

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-0.2-0.2-0.2-0.3+0.8-0.2-0.2-0.2-0.3=-1.0

0.9					



Convolutions

- Formally:
  - k x k co specified by a k x k matrix. Often k is odd.
  - Convolu e" convolution matrix across input.

-1	-1	-1
-1	8	-1
-1	-1	-1

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

0.9	-1				



Convolutions

- Formally:
  - k x k co specified by a k x k matrix. Often k is odd.
  - Convolu e" convolution matrix across input.

-1	-1	-1
-1	8	-1
-1	-1	-1

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-0.2-0.2-0.2-0.1+1.6-0.2-0.2-0.3-0.3=-0.1

0.9	-1				



Convolutions

- Formally:
  - k x k convolution specified by a k x k matrix. Often k is odd.
  - Convolution: ”Slide” convolution matrix across input.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-1	-1	-1
-1	8	-1
-1	-1	-1

0.9	-1	-.1			



Convolutions

- Formally:
  - k x k convolution specified by a k x k matrix. Often k is odd.
  - Convolution: ”Slide” convolution matrix across input.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-1	-1	-1
-1	8	-1
-1	-1	-1

0.9	-1	-.1	-.2	0.1	-.7



Convolutions

- Formally:
  - k x k convolution specified by a k x k matrix. Often k is odd.
  - Convolution: ”Slide” convolution matrix across input.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-1	-1	-1
-1	8	-1
-1	-1	-1

0.9	-1	-.1	-.2	0.1	-.7
0.1					



Convolutions

- Formally:
  - k x k convolution specified by a k x k matrix. Often k is odd.
  - Convolution: ”Slide” convolution matrix across input.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-1	-1	-1
-1	8	-1
-1	-1	-1

0.9	-1	-.1	-.2	0.1	-.7
0.1	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	4.7



Convolutions

- Formally:
  - k x k convolution specified by a k x k matrix. Often k is odd.
  - Convolution: ”Slide” convolution matrix across input.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-1	-1	-1
-1	8	-1
-1	-1	-1

Edge Detector:

Checks if middle pixel/neuron is very different from average.

0.9	-1	-.1	-.2	0.1	-.7
0.1	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	4.7



Convolutions

- Formally:
  - k x k convolution specified by a k x k matrix. Often k is odd.
  - Convolution: ”Slide” convolution matrix across input.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-1	-1	-1
-1	8	-1
-1	-1	-1

Result:

New image with “higher-level” features. Edges rather than individual colors.

0.9	-1	-.1	-.2	0.1	-.7
0.1	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	4.7



Convolutions

- Formally:
  - k x k convolution specified by a k x k matrix. Often k is odd.
  - Convolution: ”Slide” convolution matrix across input.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-1	-1	-1
-1	8	-1
-1	-1	-1

Odd k:

Means can center on a pixel.

0.9	-1	-.1	-.2	0.1	-.7
0.1	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	4.7



Convolutions

- Formally:
  - k x k convolution specified by a k x k matrix. Often k is odd.
  - Convolution: ”Slide” convolution matrix across input.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-1	-1	-1
-1	8	-1
-1	-1	-1

Translation invariant:

An edge is an edge, no matter where.

Looks for same patterns across image.

0.9	-1	-.1	-.2	0.1	-.7
0.1	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	4.7



Convolutions

- Formally:
  - k x k convolution specified by a k x k matrix. Often k is odd.
  - Convolution: ”Slide” convolution matrix across input.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

-1	-1	-1
-1	8	-1
-1	-1	-1

Annoying:

Image shrinks.

0.9	-1	-.1	-.2	0.1	-.7
0.1	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	…
…	…	…	…	…	4.7



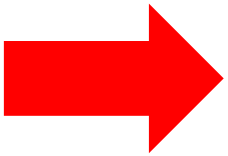
Convolutions

Can center exactly on every pixel of input image.

Same output size.

- Padding:
  - If perform convolution with k x k for odd k, pad input image with frame of (k-1)/2 0's.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0



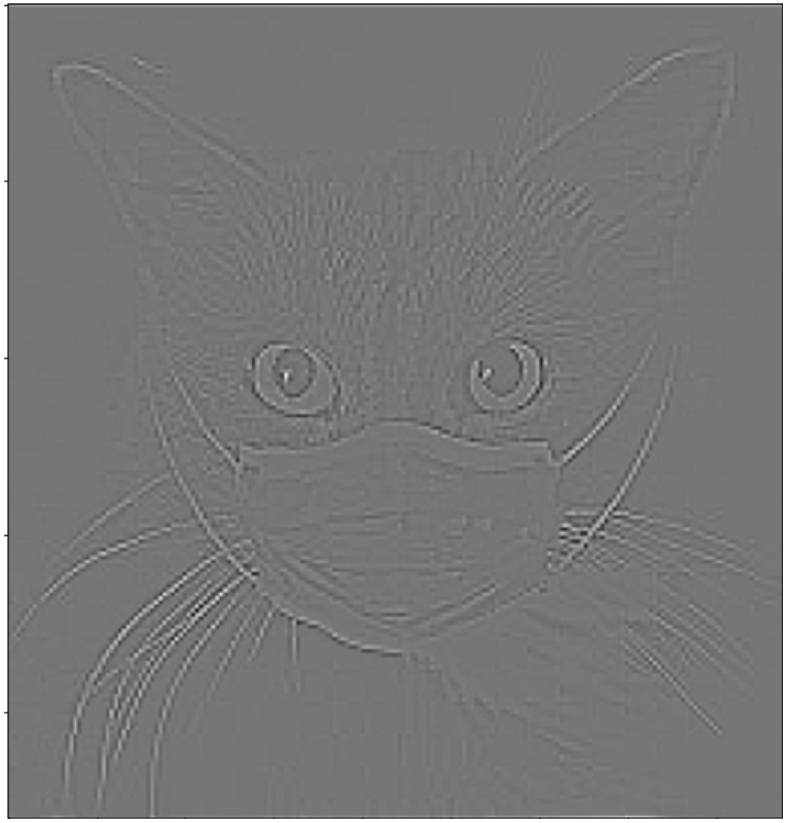
0	0	0	0	0	0	0	0	0	0
0	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2	0
0	0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2	0
0	0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2	0
0	0.2	0.5	0	0.7	0.1	0.3	0.1	0.1	0
0	0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1	0
0	0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1	0
0	0.1	0.1	0.1	0.1	0.1	1	0.9	0	0
0	0.1	0.1	0.1	0.1	0.1	0.2	0.3	0	0
0	0	0	0	0	0	0	0	0	0



Convolutions



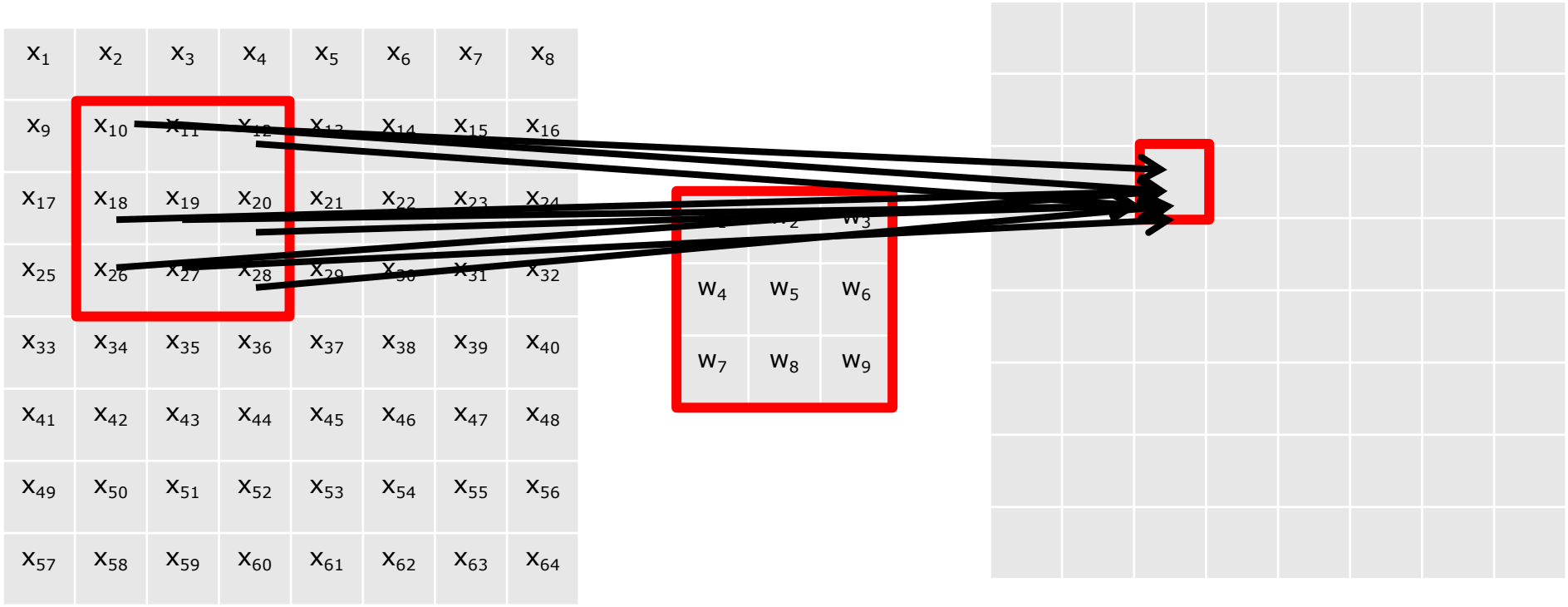
-1	-1	-1
-1	8	-1
-1	-1	-1



Convolutions

- In Neural Net:
  - Convolution has trainable parameters.

Neuron has just 9 incoming edges.



Convolutions

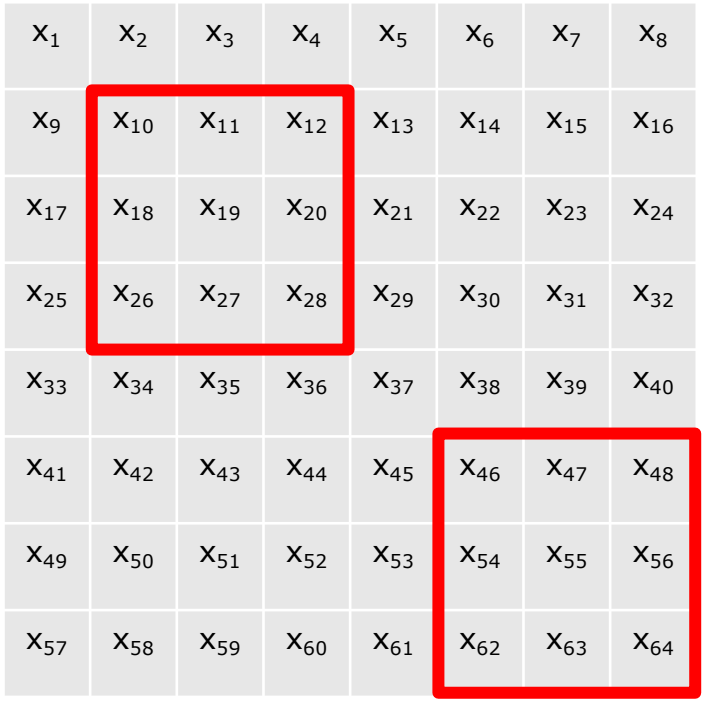
Sharing of weights.

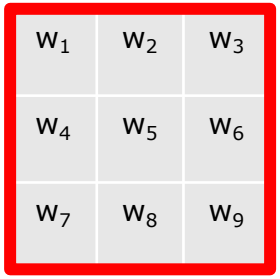
Backprop: No problem, just sum gradients along out-edges.

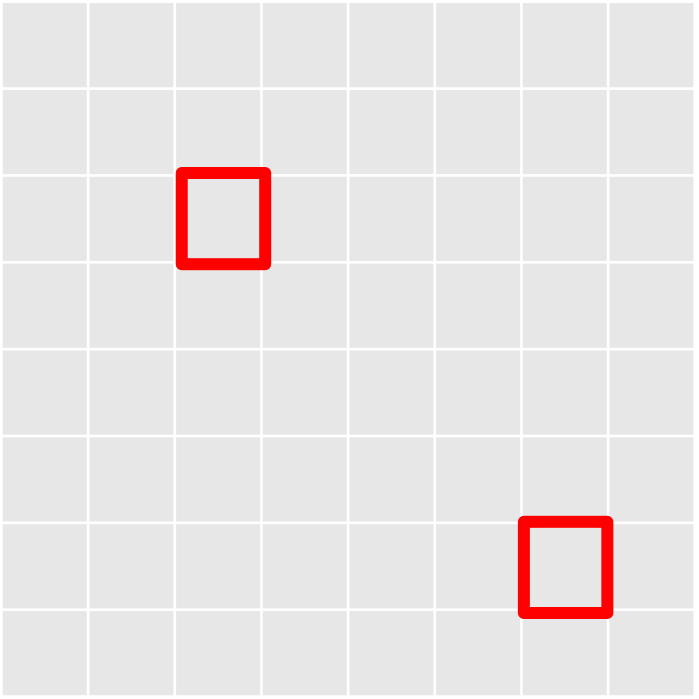
- In Neural Net:
  - Convolution has trainable parameters.

Signal:

$$w_1x_{10}+w_2x_{11}+w_3x_{12}+w_4x_{18}+w_5x_{19}+w_6x_{20}+w_7x_{26}+w_8x_{27}+w_9x_{28}$$





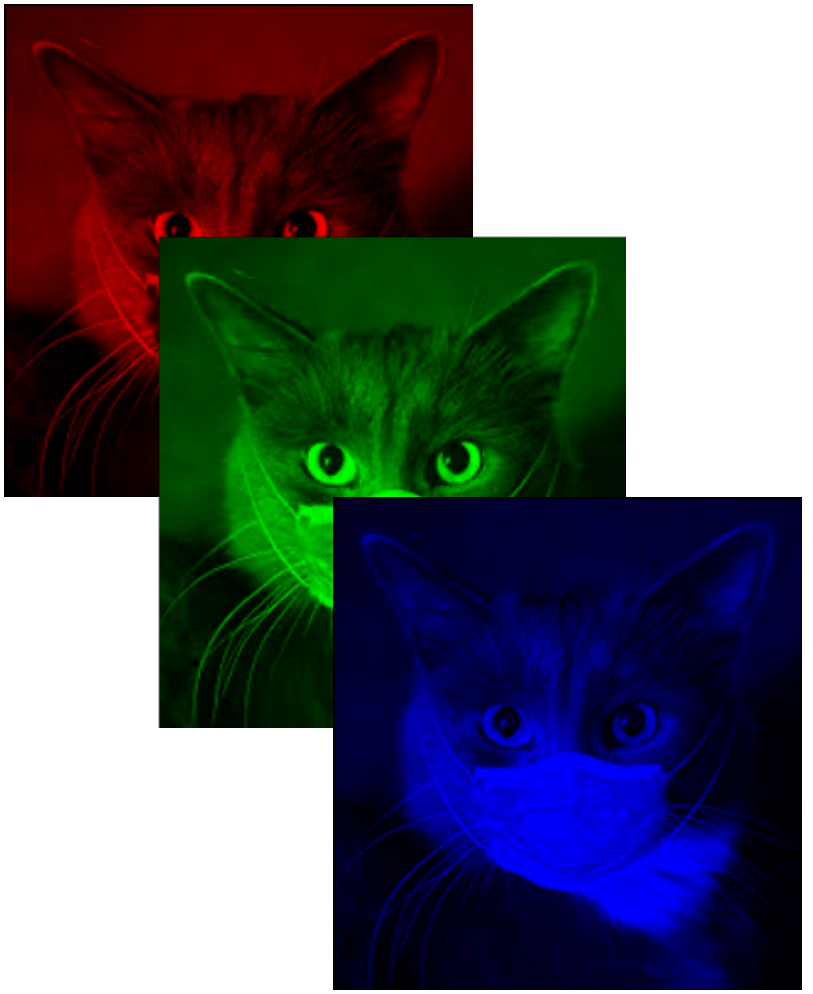


Signal:

$$w_1x_{46}+w_2x_{47}+w_3x_{48}+w_4x_{54}+w_5x_{55}+w_6x_{56}+w_7x_{62}+w_8x_{63}+w_9x_{64}$$

Channels



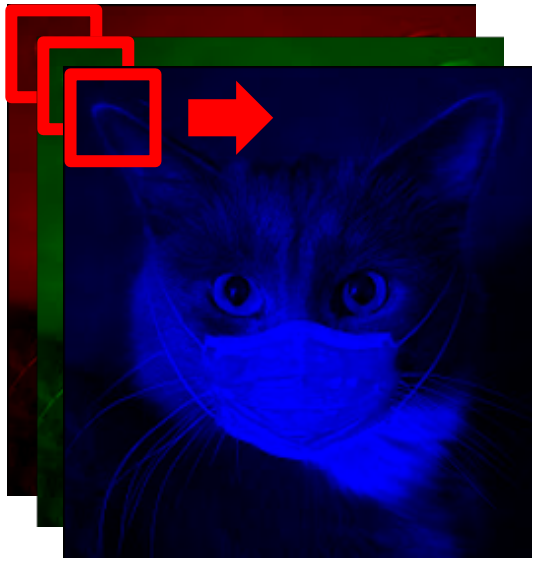


One channel per color Red, Green Blue.

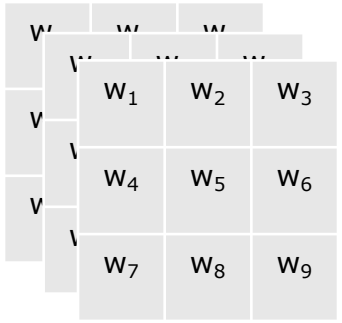
Each a number from 0=Black to 1=Red/Green/Blue

Convolutions and Channels

- Image:
  - Think of as w x h x 3 grid.
  - Last dimension gives channel among Red, Green, Blue.

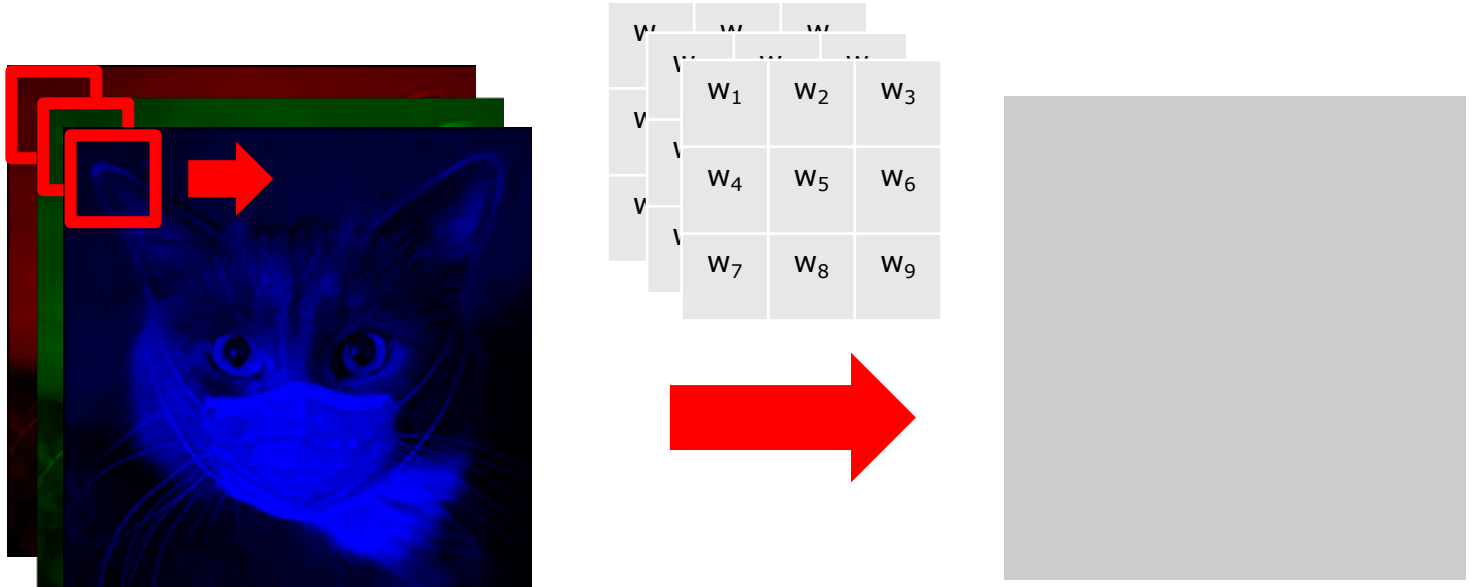


- Convolutions:
  - If c channels, then convolution is k x k x c matrix/tensor of weights.



Convolutions and Channels

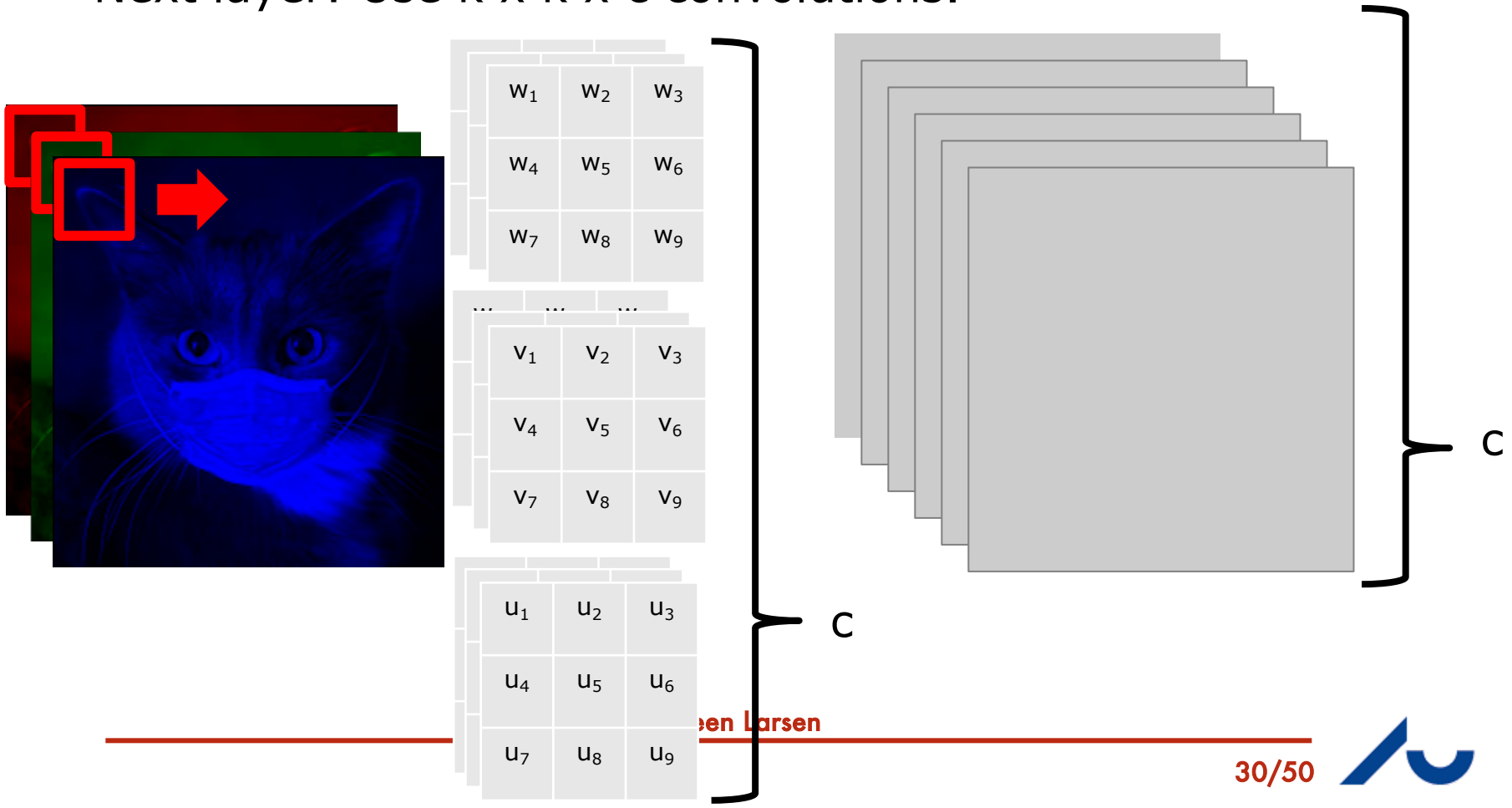
- Image:
  - Think of as w x h x 3 grid.
  - Last dimension gives channel among Red, Green, Blue.



Produces one w x h output channel

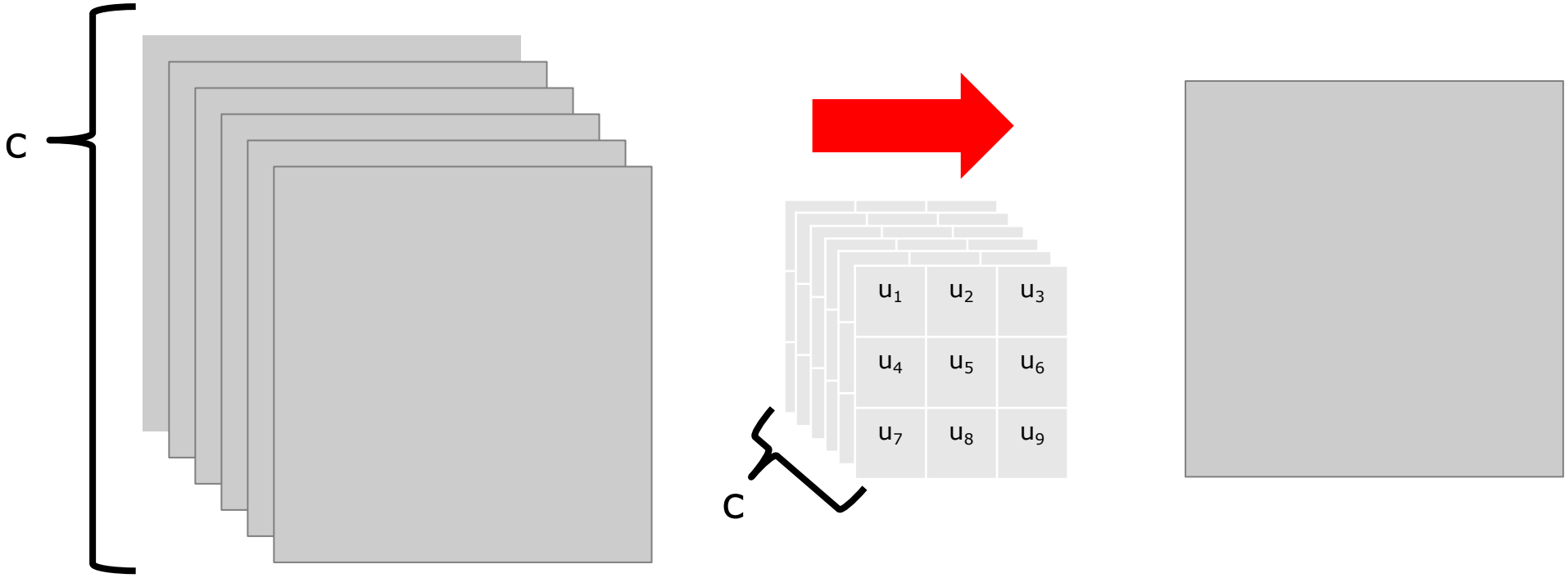
Convolutions and Channels

- Convolutions:
  - Run c convolutions producing c output channels.
  - Next layer: Use k x k x c convolutions.



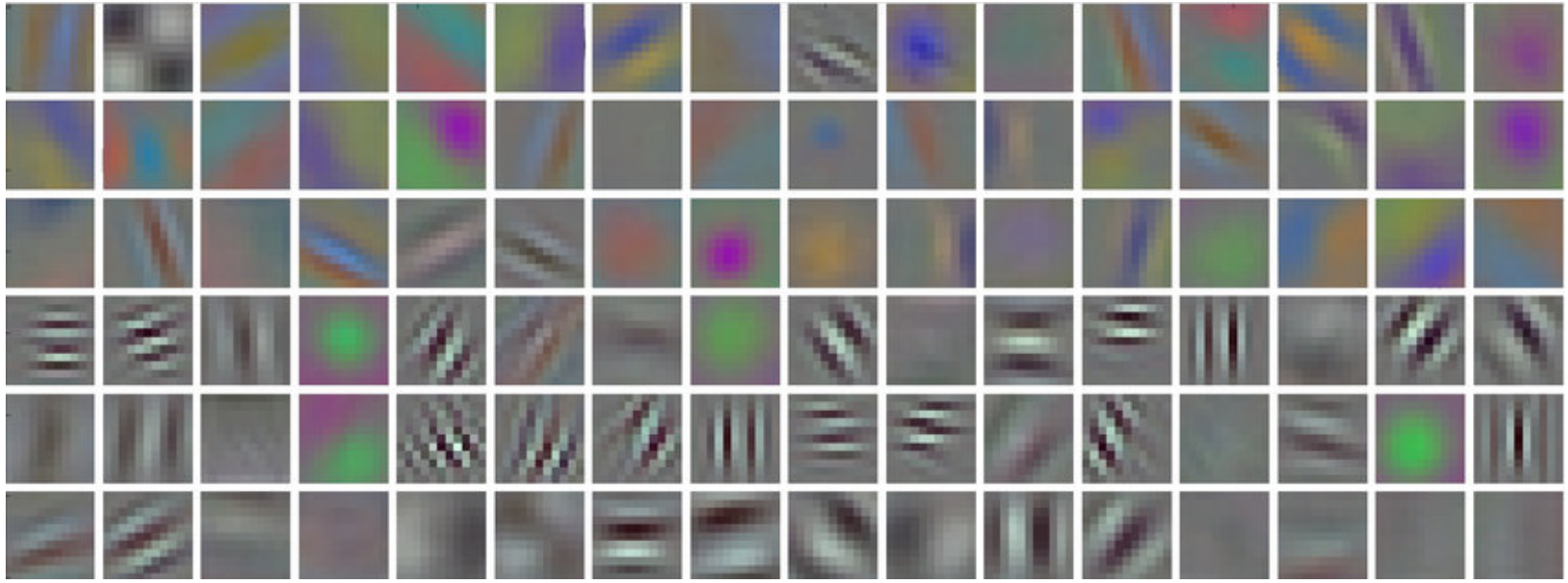
Convolutions and Channels

- Convolutions:
  - Run c convolutions producing c output channels.
  - Next layer: Use k x k x c convolutions.



Convolutions and Channels

- First layer convolutions learned by a convolutional neural net for image recognition.
  - Here the three channels have been collapsed for visualization.



Each convolution gives a channel in next layer, so 90 channels



Convolutions

- Idea:
  - Look for small features in the image.
  - Should be translation invariant.
  - Preserve information on feature's position in image.



Cat, near right side



Cat, near left side

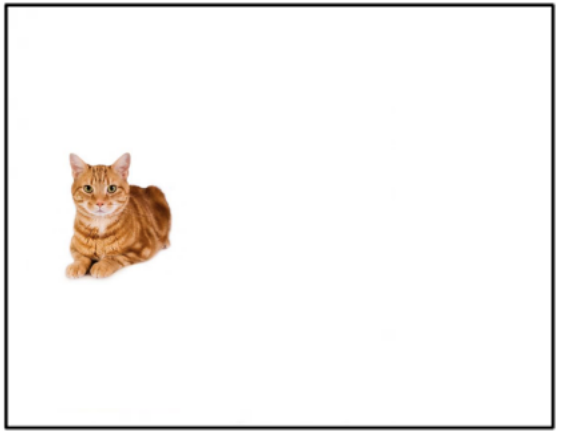
- Problem: Not really scale invariant

Convolutions

- Idea:
  - Look for small features in the image.
  - Should be translation invariant.
  - Preserve information on feature's position in image.



Cat, near right side



Cat, near left side

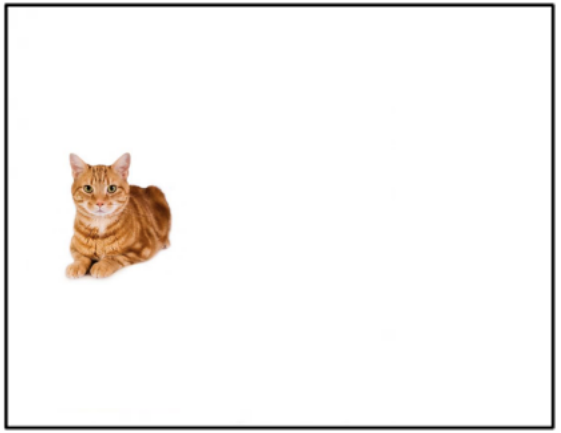
- Problem: Not really scale invariant

Pooling

- Idea:
  - Zoom out on image and ”focus” on bigger ”features.



Cat, near right side



Cat, near left side

- Problem: Not really scale invariant

Pooling

- Formally:
  - k x k pooling specified by integer k and function of k elements $\phi : R^k \rightarrow R$. Stride s.
  - Popular examples: ϕ = max, mean, min

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

max

Stride 2



Pooling

- Formally:
  - k x k pooling specified by integer k and function of k elements $\phi : R^k \rightarrow R$. Stride s.
  - Popular examples: ϕ = max, mean, min

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

max

Stride 2

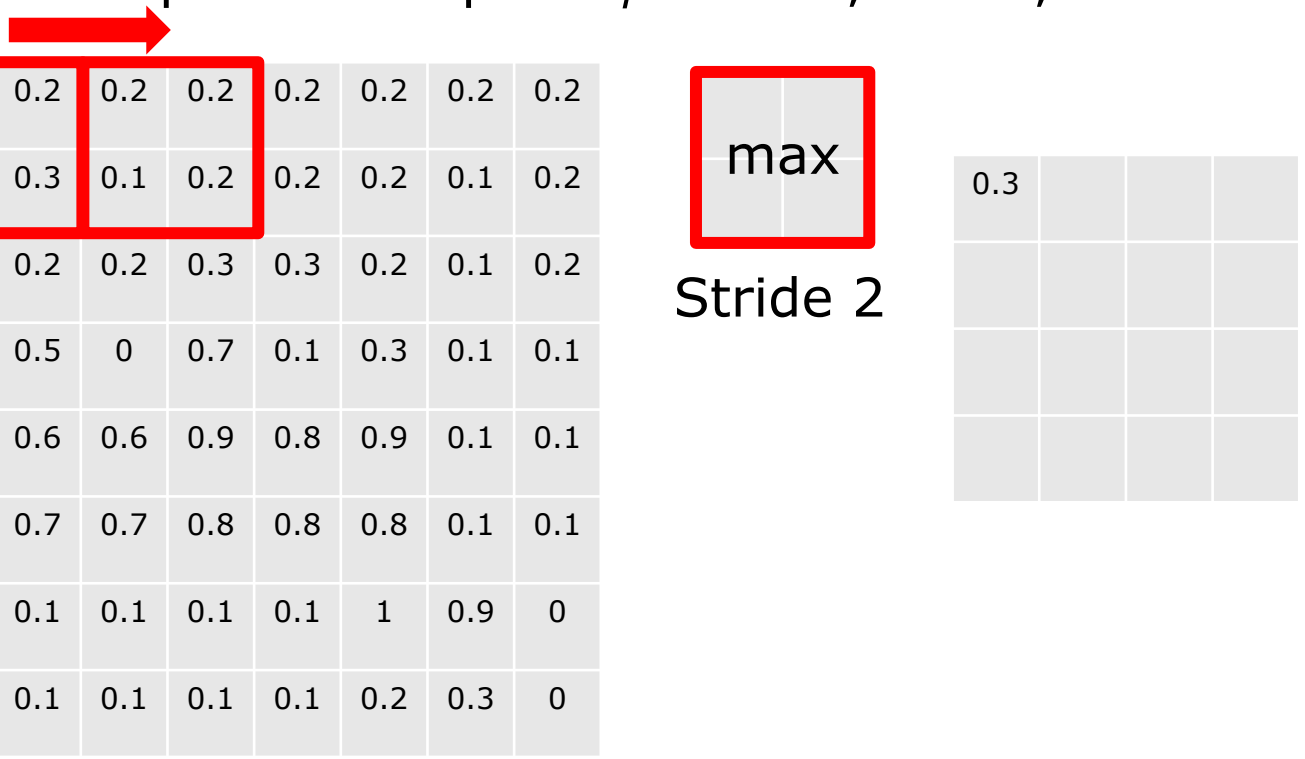
0.3			



Pooling

- Formally:
  - k x k pooling specified by integer k and function of k elements $\phi : R^k \rightarrow R$. Stride s.
  - Popular examples: ϕ = max, mean, min

Moves by *stride*



Pooling

- Formally:
  - k x k pooling specified by integer k and function of k elements $\phi : R^k \rightarrow R$. Stride s.
  - Popular examples: ϕ = max, mean, min

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

max

Stride 2

0.3			



Pooling

- Formally:
  - k x k pooling specified by integer k and function of k elements $\phi : R^k \rightarrow R$. Stride s.
  - Popular examples: ϕ = max, mean, min

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

max

Stride 2

0.3	0.2		



Pooling

- Formally:
  - k x k pooling specified by integer k and function of k elements $\phi : R^k \rightarrow R$. Stride s.
  - Popular examples: ϕ = max, mean, min

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

max

Stride 2

0.3	0.2		



Pooling

- Formally:
  - k x k pooling specified by integer k and function of k elements $\phi : R^k \rightarrow R$. Stride s.
  - Popular examples: ϕ = max, mean, min

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

max

Stride 2

0.3	0.2	0.2	



Pooling

- Formally:
  - k x k pooling specified by integer k and function of k elements $\phi : R^k \rightarrow R$. Stride s.
  - Popular examples: ϕ = max, mean, min

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

max

Stride 2

0.3	0.2	0.2	0.2



Pooling

- Formally:
  - k x k pooling specified by integer k and function of k elements $\phi : R^k \rightarrow R$. Stride s.
  - Popular examples: ϕ = max, mean, min

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

max

Stride 2

0.3	0.2	0.2	0.2



Pooling

- Formally:
  - k x k pooling specified by integer k and function of k elements $\phi : R^k \rightarrow R$. Stride s.
  - Popular examples: ϕ = max, mean, min

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

max

Stride 2

0.3	0.2	0.2	0.2
0.5			



Pooling

- Formally:
  - k x k pooling specified by integer k and function of k elements $\phi : R^k \rightarrow R$. Stride s.
  - Popular examples: ϕ = max, mean, min

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

max

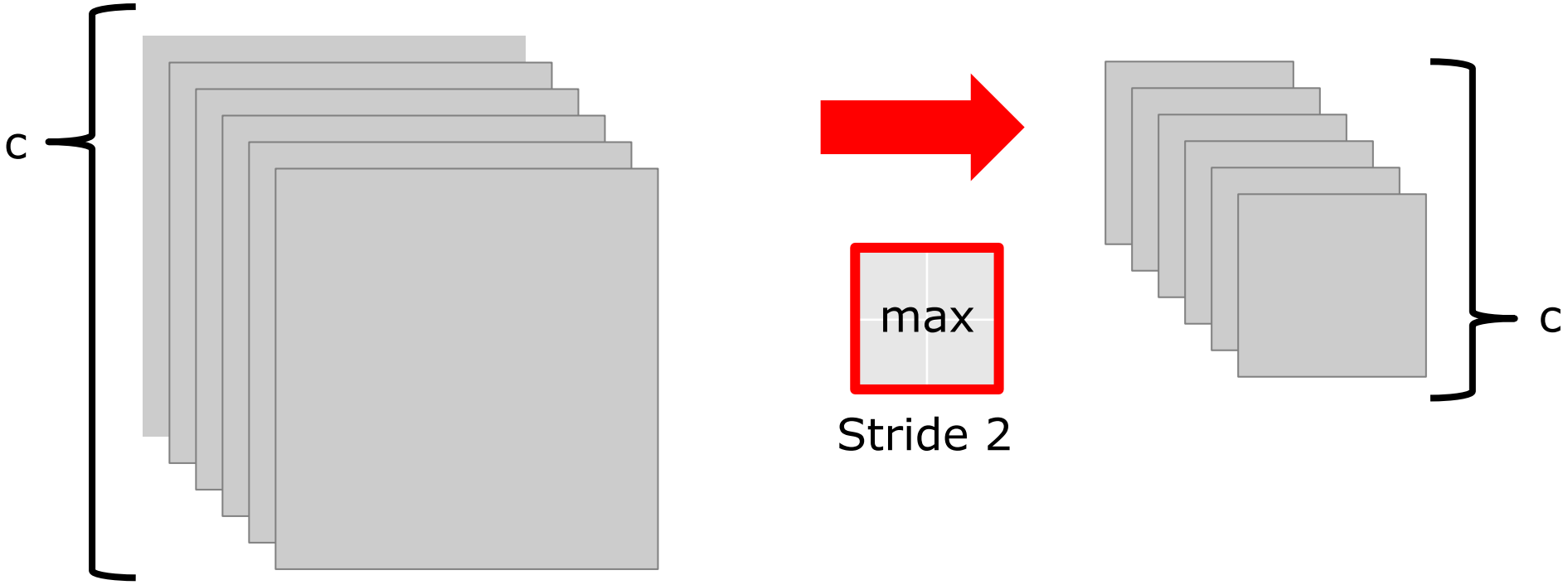
Stride 2

0.3	0.2	0.2	0.2
0.5	0.7	0.3	0.2
0.7	0.9	0.9	0.1
0.1	0.1	1	0.9



Channels and Pooling

- Pooling:
  - Run pooling operation on each channel individually



Pooling

- Zooming Out:
  - Pooling with stride s>1 effectively reduces image size by factor s.
  - Zooms out.

0.2	0.2	0.2	0.2	0.2	0.2	0.2	0.2
0.2	0.3	0.1	0.2	0.2	0.2	0.1	0.2
0.2	0.2	0.2	0.3	0.3	0.2	0.1	0.2
0.2	0.5	0	0.7	0.1	0.3	0.1	0.1
0.2	0.6	0.6	0.9	0.8	0.9	0.1	0.1
0.2	0.7	0.7	0.8	0.8	0.8	0.1	0.1
0.1	0.1	0.1	0.1	0.1	1	0.9	0
0.1	0.1	0.1	0.1	0.1	0.2	0.3	0

max

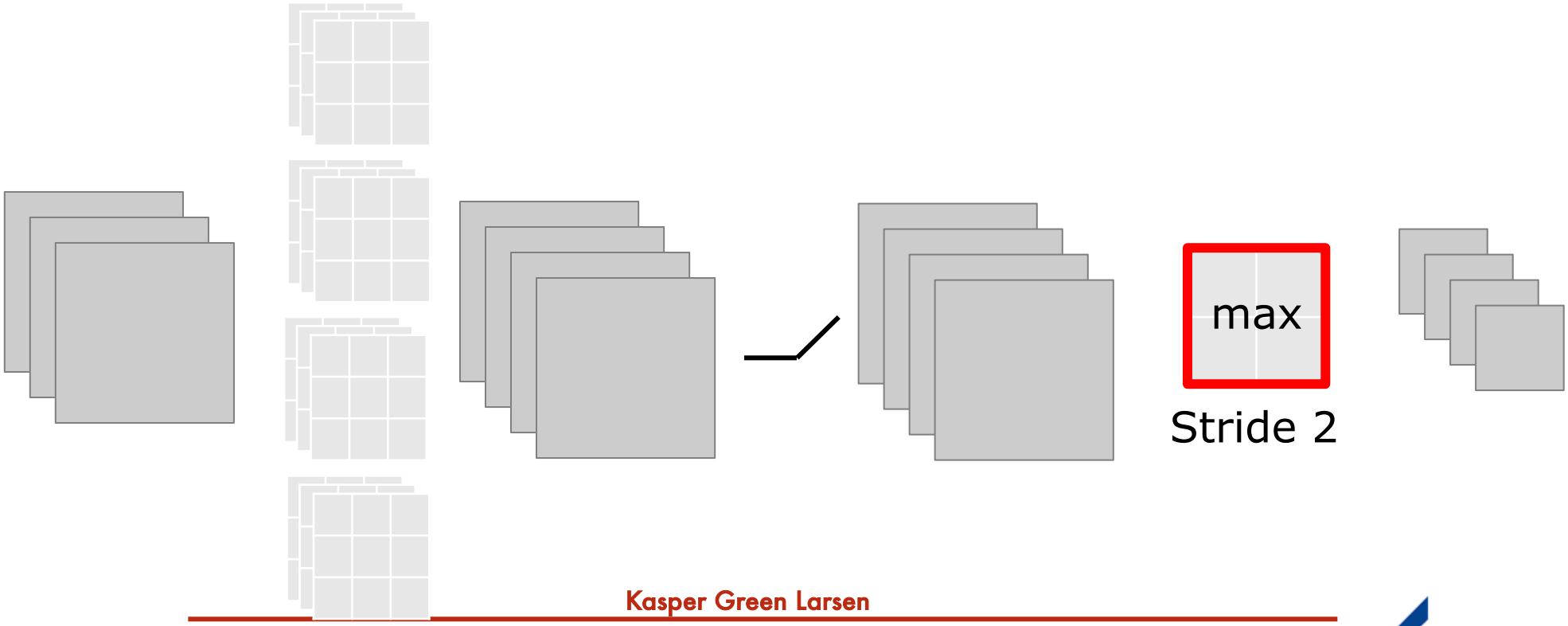
Stride 2

0.3	0.2	0.2	0.2
0.5	0.7	0.3	0.2
0.7	0.9	0.9	0.1
0.1	0.1	1	0.9



Convolution + Activation + Pooling

- Standard Deep Net Block:
  - A number of convolutions c.
  - Followed by activation function, such as ReLU.
  - Followed by pooling, such as max with stride 2.



Deep Nets for Images

- Repeat a block multiple times
- End with fully connected (and softmax for classification)

